

OpenGL 프로그래밍

동명대학교 게임공학과

강영민

OpenGL과 glfw 설치

```
% pip install PyOpenGL glfw
```



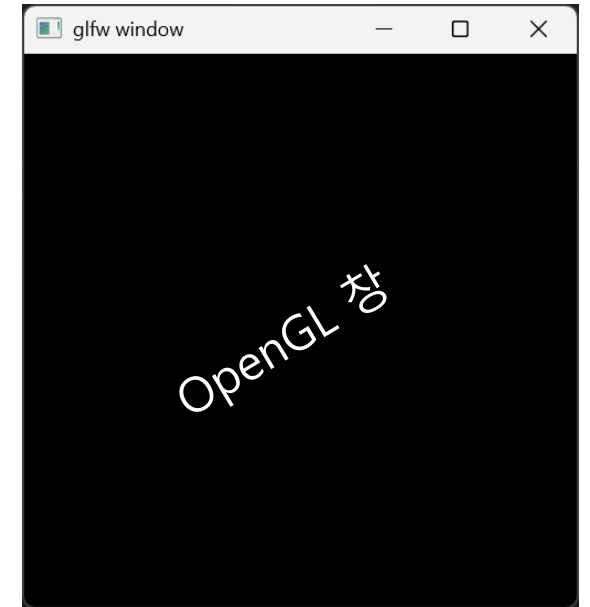
OpenGL을 사용할 수 있는 시스템 환경 구축이 완료됨

OpenGL 윈도우 생성하기 위한 구조

```
def graphic_code():  
    pass
```

```
def main():  
    window = initialize_window(500, 500, "glfw window")  
  
    main_loop(window, graphic_code)
```

```
if __name__ == "__main__":  
    main()
```



main 함수를 진입점을 사용하는 형태로 코딩

필요한 함수 구현

```
import glfw
from OpenGL.GL import *

#####

def initialize_window(w, h, window_title):
    if not glfw.init():
        return None
    window = glfw.create_window(w, h, window_title, None, None)
    if not window:
        glfw.terminate()
        print("window initialization failed")
        exit(0)

    glfw.make_context_current(window)
    return window

def main_loop(window, graphic_code):
    while not glfw.window_should_close(window):
        graphic_code()
        glfw.swap_buffers(window)
        glfw.poll_events()

    glfw.destroy_window(window)
    glfw.terminate()

#####
```

OpenGL 동작의 기본적인 분류

- 초기화^{initialize} – OpenGL 콘텐츠의 초기화 콜백
- 크기 변경^{resize} – 윈도우가 생성되거나 크기가 변경될 때 호출되는 콜백
- 디스플레이^{display} – 화면에 그림을 그리는 일이 필요할 때 호출되는 콜백

```
def initialize(window):  
    # OpenGL initialization code here (e.g., glClearColor, etc.)  
    glClearColor(0.0, 0.0, 0.0, 1.0)  
    # Additional setup as needed
```

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
  
    # Drawing code here  
    # For example: glBegin(GL_TRIANGLES); ... glEnd()  
    pass
```

```
def reshape(window, width, height):  
    glViewport(0, 0, width, height)  
    # Additional reshape logic as needed (e.g., projection matrix)
```

Main 함수의 변경

- Reshape은 glfw에 등록, Initialize 코드 실행, 매 프레임 반복적으로 실행할 display 코드는 루프에 전달

```
def main():  
    window = initialize_window(500, 500, "glfw window")  
  
    # Set up callbacks similar to GLUT  
    glfw.set_framebuffer_size_callback(window, reshape)  
  
    # Call initialize  
    initialize(window)  
  
    # Main loop calling display  
    main_loop(window, display)
```

그림 그리기

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)  
  
    glBegin(GL_TRIANGLES)  
        glColor3f(1, 1, 0)  
        glVertex3f(-1, 0, 0)  
        glVertex3f( 1, 0, 0)  
        glVertex3f( 0, 1, 0)  
        glColor3f(0, 1, 1)  
        glVertex3f(-1, 0.5, 0)  
        glVertex3f( 1, 0.5, 0)  
        glVertex3f( 0, -0.5, 0)  
    glEnd()
```



You are now a graphics programmer!

Primitives

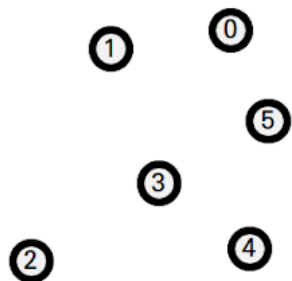
- OpenGL이 제공하는 그리기 기본 요소
 - 정점 데이터를 어떻게 조합할 것인지를 결정

프리티미브 사용 방법
glBegin(프리티미브 지정) 정점 정보 제공(glVertex3f 등을 이용) glEnd()

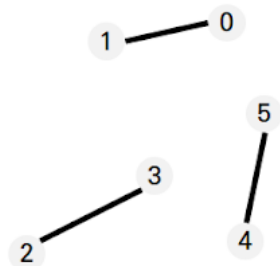
Primitive의 종류

- GL_POINTS – 입력된 정점을 하나씩 점으로 가시화
- GL_LINES – 입력된 정점을 두 개씩 묶어 선분으로 표현
- GL_LINE_STRIP – 입력된 정점을 차례로 연결하여 하나의 폴리라인^{polyline} 구성
- GL_LINE_LOOP – 입력된 정점을 차례로 연결한 뒤에 마지막 점을 시작점으로 연결
- GL_TRIANGLES – 입력된 정점을 세 개씩 묶어 삼각형을 그림
- GL_TRIANGLE_STRIP – 처음 세 정점으로 삼각형 그린 뒤, 정점 추가될 때마다 삼각형을 직전 두 개 정점과 연결
- GL_TRIANGLE_FAN – 부채 모양으로 삼각형을 추가해 나감
- GL_QUADS – 정점 네 개씩을 묶어 사각형 그리기
- GL_QUAD_STRIP: 처음 네 개 정점으로 사각형 그리고, 이후 두 개씩 묶어 직전 두 개 정점과 함께 사각형 그리기
- GL_POLYGON: 입력된 모든 정점으로 다각형을 그림

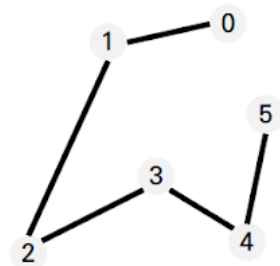
Primitives



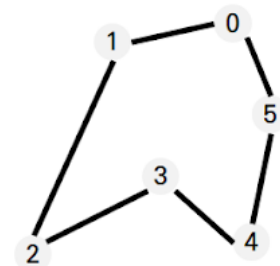
GL_POINTS



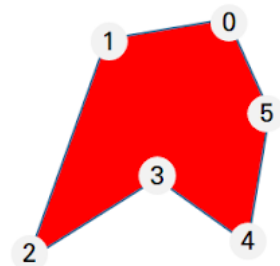
GL_LINES



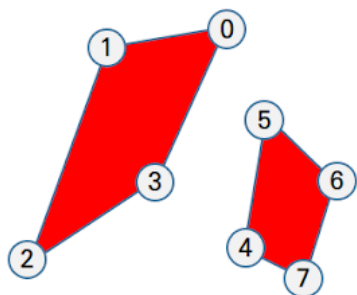
GL_LINE_STRIP



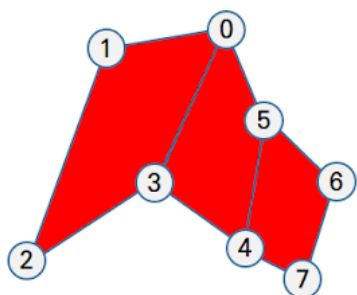
GL_LINE_LOOP



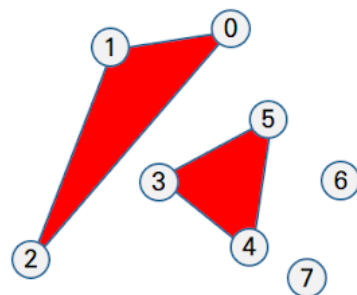
GL_POLYGON



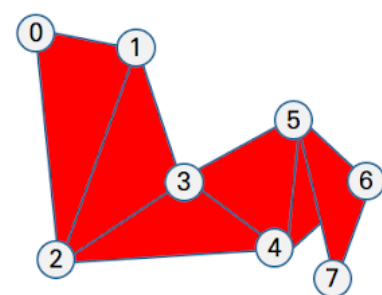
GL_QUADS



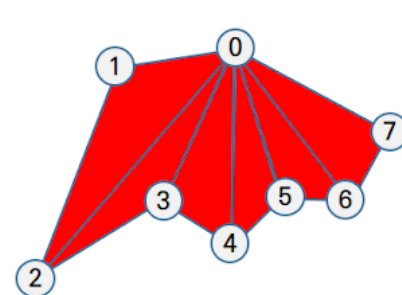
GL_QUAD_STRIP



GL_TRIANGLES



GL_TRIANGLE_STRIP



GL_TRIANGLE_FAN

정점 정보 제공 방법

정점 정보 제공 방법

```
ix, iy, iz = -1, 0, 0  
fx, fy, fz = 1.0, 0.0, 0.0  
fverts = [fx-1.0, fy+1.0, fz]
```

```
glBegin(GL_TRIANGLES)  
glVertex3i(ix, iy, iz)  
glVertex3f(fx, fy, fz)  
glVertex3fv(fverts)  
glEnd()
```

주요 정점 정보

- 대표적인 것은 정점의 위치, 해당 정점에서 면의 **법선**normal vector, 그리고 색상 등
- 위의 예는 정점의 위치만 제공

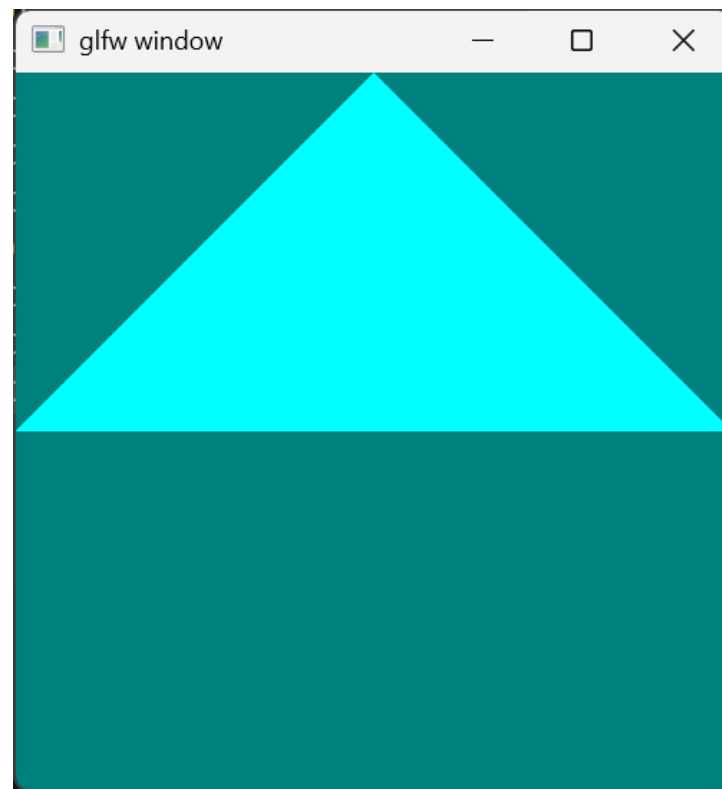
간단한 그리기 연습

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

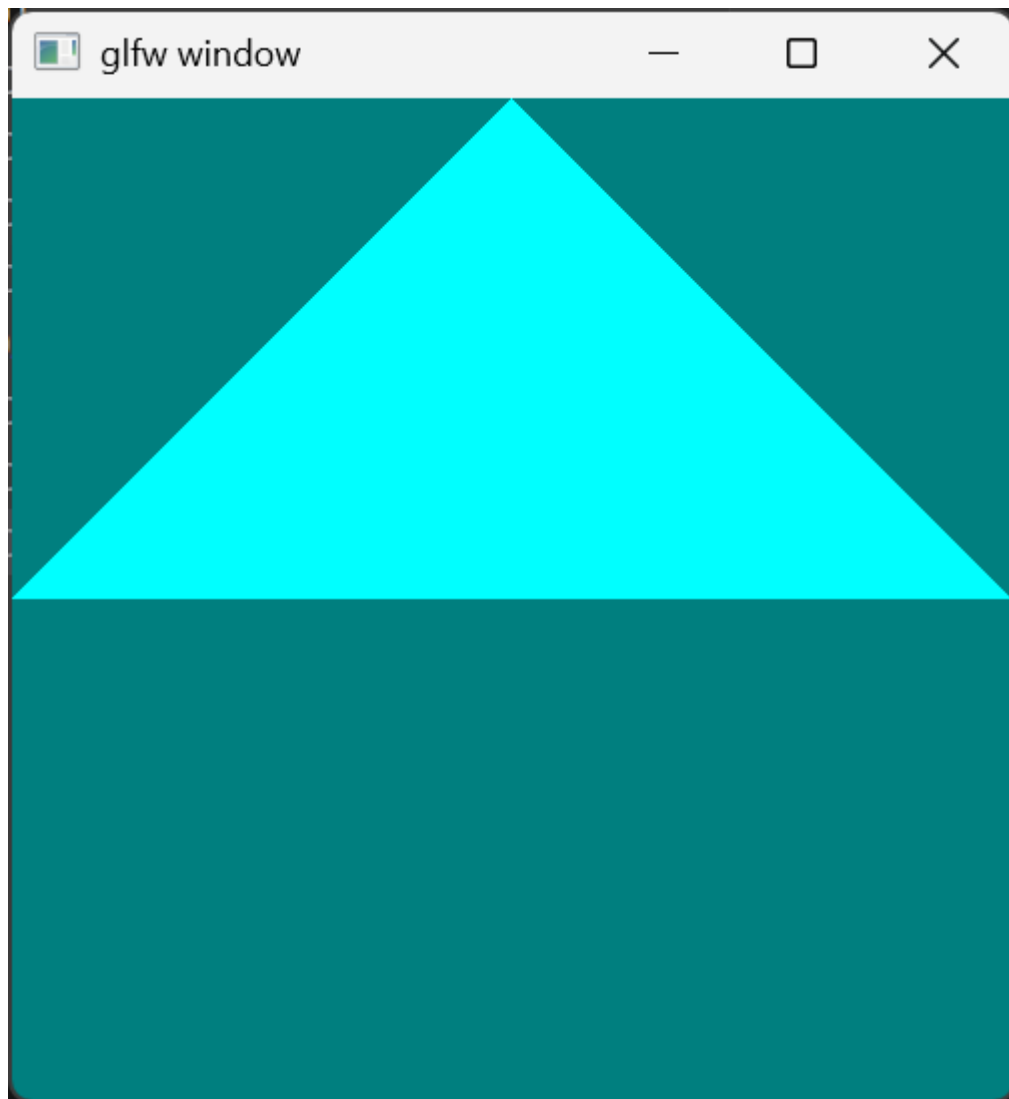
```
    glMatrixMode(GL_MODELVIEW)  
    glLoadIdentity()
```

```
    ix, iy, iz = -1, 0, 0  
    fx, fy, fz = 1.0, 0.0, 0.0  
    fverts = [fx-1.0, fy+1.0, fz]
```

```
    glColor3f(0, 1, 1)  
    glBegin(GL_TRIANGLES)  
    glVertex3i(ix, iy, iz)  
    glVertex3f(fx, fy, fz)  
    glVertex3fv(fverts)  
    glEnd()
```



결과



정점 정보 제공시 사용한 함수

- 정수 좌표를 제공하기 위해 `glVertex3i`
- 부동소수점으로 표시된 좌표 `glVertex3f`
- `v`가 접미사로 붙은 함수
 - 이것은 벡터 `vector`를 의미
 - 정점 정보가 배열이나 리스트와 같은 묶음
 - `3fv` = 3차원 데이터, 부동소수점, 묶음

Project 1

마우스 클릭을 이용하여 정점 입력하기

정점 초기화

```
points = [] # Global list to store points
```

마우스 처리 함수 등록

```
def main():  
    window = initialize_window(500, 500, "glfw window")  
  
    # Set up callbacks similar to GLUT  
    glfw.set_framebuffer_size_callback(window, reshape)  
    glfw.set_mouse_button_callback(window, mouse_callback)  
  
    # Call initialize  
    initialize(window)  
  
    # Main loop calling display  
    main_loop(window, display)
```

Project 1

마우스 클릭을 이용하여 정점 입력하기

마우스 처리 함수

```
def mouse_callback(window, button, action, mods):  
    global points  
    if button == glfw.MOUSE_BUTTON_LEFT and action == glfw.PRESS:  
        mx, my = glfw.get_cursor_pos(window)  
        width, height = glfw.get_framebuffer_size(window)  
        # Normalize to [-1, 1] range, accounting for y-inversion  
        nx = 2.0 * (mx / width) - 1.0  
        ny = 1.0 - 2.0 * (my / height)  
        points.append([nx, ny, 0.0])  
        print(f"mouse clicked at: ({mx}, {my})")  
        print(f"Point added: ({nx:.2f}, {ny:.2f}, 0.0)")
```

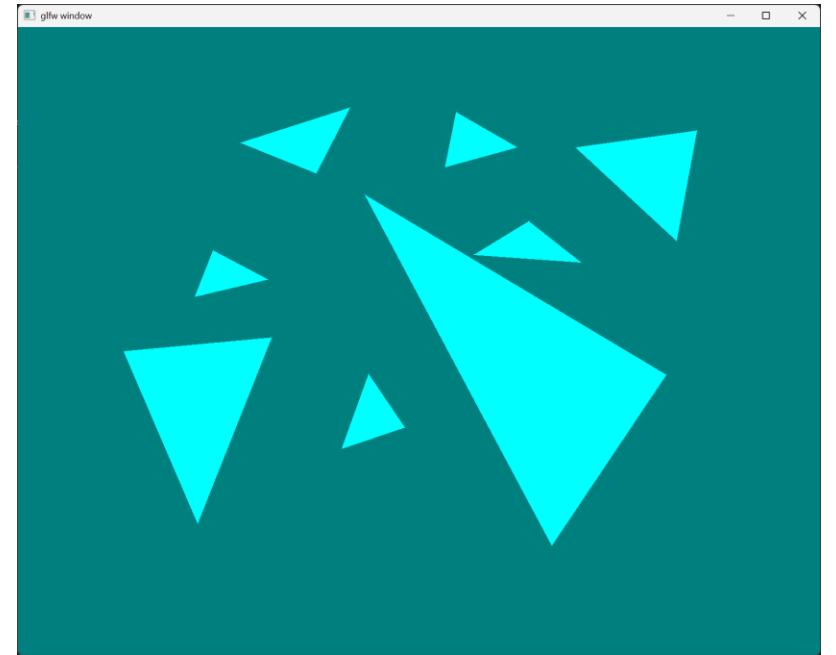
Project 1

마우스 클릭을 이용하여 정점 입력하기

입력된 정점을 그리자

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
    glBegin(GL_POINTS)  
    for point in points:  
        glVertex3fv(point)  
    glEnd()  
  
    if len(points) >= 3:  
        glColor3f(0, 1, 1)  
        glBegin(GL_TRIANGLES)  
        for point in points:  
            glVertex3fv(point)  
        glEnd()
```



Project 2

다양한 프리미티브를 적용해 보기

프리미티브 변수

```
points = [] # Global list to store points
current_primitive = GL_POINTS # Default primitive
```

키보드 함수 등록

```
def main():
    window = initialize_window(500, 500, "glfw window")

    # Set up callbacks similar to GLUT
    glfw.set_framebuffer_size_callback(window, reshape)
    glfw.set_mouse_button_callback(window, mouse_callback)
    glfw.set_key_callback(window, key_callback)
```

Project 2

다양한 프리미티브를 적용해 보기

키보드 콜백

```
def key_callback(window, key, scancode, action, mods):
    global current_primitive
    if action == glfw.PRESS:
        if glfw.KEY_0 <= key <= glfw.KEY_9:
            primitive_index = key - glfw.KEY_0
            primitives = [
                GL_POINTS,
                GL_LINES,
                GL_LINE_STRIP,
                GL_LINE_LOOP,
                GL_TRIANGLES,
                GL_TRIANGLE_STRIP,
                GL_TRIANGLE_FAN,
                GL_QUADS,
                GL_QUAD_STRIP,
                GL_POLYGON
            ]
            if primitive_index < len(primitives):
                current_primitive = primitives[primitive_index]
                print(f"Selected primitive {primitive_index}: {current_primitive}")
```

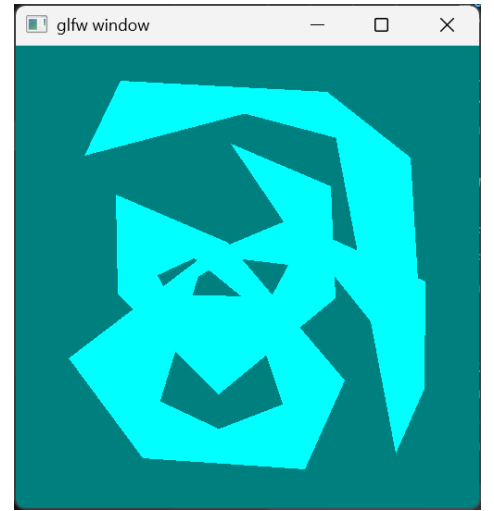
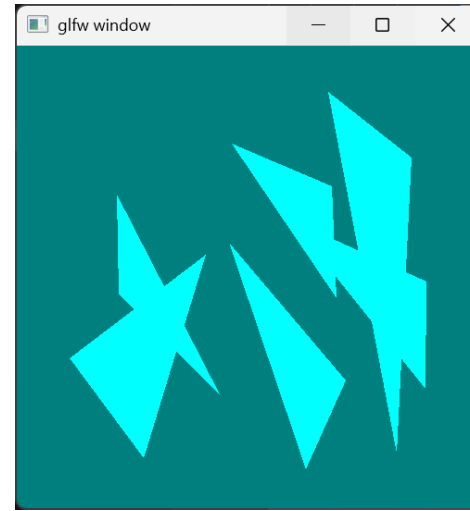
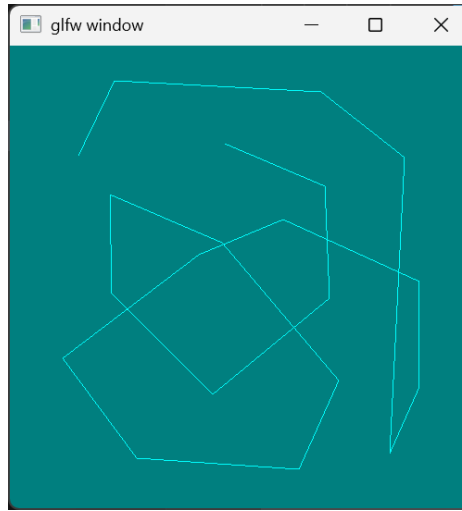
Project 2

다양한 프리미티브를 적용해 보기

그리기 콜백

```
def display(window):  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
    glColor3f(0, 1, 1)  
    glBegin(current_primitive)  
    for point in points:  
        glVertex3fv(point)  
    glEnd()
```



열심히 만들어 봅시다!